

Reporte de implementación.

Mauricio Olguín Sánchez - A01711522

24 de agosto de 2026

Índice general

1. Realizando la implementación «A manita»

- 1.1. Sobre la base de datos seleccionada
- 1.2. Proceso de ETL
 - 1.2.1. *Extraction*
 - 1.2.2. *Transforming*
 - 1.2.3. *Loading*

2. Anexo

- 2.1. Códigos

Capítulo 1

Realizando la implementación «Amanita»

1.1. Sobre la base de datos seleccionada

Después de haber investigado en páginas como [UC Irvine](#), [Kaggle](#), el [World Bank Open Data](#), entre otros, encontré en la página de [OpenML](#) un *dataset* interesante. Tras aplicar varios filtros, encontré la base de datos [Airlines_DepDelay_10M](#).

Esta base de datos está disponible en OpenML y está construida a partir de datos provenientes de la Oficina de Estadísticas de Transporte de los Estados Unidos (*Bureau of Transportation Statistics*, BTS), específicamente de la base de datos *Airline On-Time Performance Data*.

La base de datos seleccionada contiene 10,000,000 de registros y 10 variables, y corresponde a un problema de regresión. A continuación, presento una breve descripción general del *dataset*:

Propósito: Predecir el tiempo de retraso en la salida de los vuelos, medido en minutos (*Departure Delay*).

Fecha de los datos: Desde 1987 hasta 2013.

Formato: ARFF (*Attribute-Relation File Format*).

Tipo de tarea: Regresión.

Número de registros: 10,000,000.

Número de variables: 10.

Aunque OpenML proporciona información sobre cada variable, dentro de la fuente original (en el [Bureau of Transportation Statistics](#)), existe una descripción más detallada sobre los datos:

DepDelay - numerico. Diferencia, en minutos, entre la hora de salida programada y la hora de salida real del vuelo. Los valores negativos indican que el vuelo salió antes de lo programado.

Month - numerico. Mes en el que se realizó el vuelo, con valores del 1 al 12, donde 1 corresponde a enero y 12 a diciembre.

DayofMonth - numerico. Día del mes en el que se realizó el vuelo, con valores del 1 al 31.

DayOfWeek - numerico. Día de la semana en el que se realizó el vuelo, representado mediante valores del 1 al 7.

CRSDepTime - numerico. Hora de salida programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

CRSArrTime - numerico. Hora de llegada programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

UniqueCarrier - categorico. Código único utilizado para identificar a la aerolínea que opera el vuelo.

Origin - categorico. Código del aeropuerto de origen del vuelo.

Dest - categorico. Código del aeropuerto de destino del vuelo.

Distance - numerico. Distancia total del vuelo, expresada en millas.

En contraste, las variables *UniqueCarrier*, *Origin* y *Dest* son explícitamente categóricas, pero serán excluidas del modelo. Su transformación (utilizando la misma técnica) produciría un incremento considerablemente mayor en la cantidad de variables. Principalmente, tomo esta decisión con el objetivo de mantener un tiempo de ejecución razonable y centrar el proyecto en la implementación manual del algoritmo de regresión.

Sobre los atributos *CRSDepTime* y *CRSArrTime*

Las variables *CRSDepTime* y *CRSArrTime* representan las horas programadas de salida y llegada de los vuelos. En el conjunto de datos, estas se representan en el formato HHMM, es decir, que si la hora era 13:20, en los datos se muestran 1320.

Sin embargo, esta representación no resulta adecuada al momento de utilizarla directamente como valores numéricos porque, dentro del modelo de regresión, no será representado la distancia real. Por ejemplo, los valores de 2355 y 5, aunque están apartados por 10 minutos, utilizando los datos tal cual como vienen en el conjunto de datos, estos estarían muy separados entre sí.

Es por esto que ambas variables tienen que ser transformadas y, la mejor conversión que he podido investigar es utilizar como referencia la medianoche.

Por esta razón, ambas variables fueron transformadas a minutos transcurridos desde la medianoche. Para realizar esta conversión, se separó la hora y los minutos del valor original y se aplicó la siguiente expresión:

$$\text{Minutos desde medianoche} = 60 \times \text{Hora} + \text{Minutos}$$

De esta forma, conservamos la información temporal original y proporcionamos una representación numérica más adecuada para el modelo de regresión.

1.2. Proceso de ETL

1.2.1. *Extraction*

Como ya lo he mencionado antes, los datos se han obtenido del siguiente link: <https://www.openml.org/search?type=data&status=active&id=42728>

Sin embargo, si se quisiera obtener más datos actuales, habría que descargarlos (sea de forma manual o creando algún *script*) desde el siguiente link: <https://transtats.bts.gov/PREZIP/>.

Aquí, se recopilan todos los datos desde 1987 hasta junio de 2026.

1.2.2. *Transforming*

Las variables *Month*, *DayofMonth* y *DayOfWeek*, aunque se encuentran representadas numéricamente en el conjunto de datos, las trataré como variables categóricas. Esto se debe a que, en realidad, los valores numéricos representan categorías y no implican necesariamente una relación lineal entre ellas. Por ejemplo, el mes representado por el valor 12 no debe interpretarse como una categoría doce veces mayor que el mes representado por el valor 1.

Para representar estas variables dentro del modelo de regresión, utilizaré la técnica de *One-Hot Encoding*. Aunque esto implicará que crezca la dimensionalidad del conjunto de datos.

1.2.3. *Loading*

Una vez hechas las transformaciones anteriores, ahora dividiré el tamaño de mi *dataset* en tres principales secciones: *train*, *validation* y *test*. Al ser la cantidad de datos tan alta, podría realizar varias particiones para *validation*, sin embargo, en esta primera parte en donde me enfoco más

Capítulo 2

Anexo

2.1. Códigos

El siguiente código sirve únicamente para la generación de los gráficos presentados, aunque no todos fueron utilizados.

```
1 # Archivo auxiliar para generar imagenes.
2
3 import matplotlib.pyplot as plt
4 import numpy as np
5 import pandas as pd
6
7 DPI = 600 # Resolución final de los gráficos
8
9 # def scatter_matrix_plot(df:pd.DataFrame):
10 #     print("Generando Scatter Matrix (solo con columnas
11 #         CRSDepTime, CRSArrTime y Distance)...")
12 #     axes = pd.plotting.scatter_matrix(df, figsize=(9, 9))
13 #     for ax in axes.flatten():
14 #         ax.xaxis.label.set_rotation(45)
15 #         ax.yaxis.label.set_rotation(0)
16 #         ax.yaxis.label.set_ha('right')
17 #     plt.tight_layout()
18 #     plt.title("")
19 #     plt.savefig("scatter_matrix.png", dpi=DPI)
20 #     print(" Gr áfico generado! guardado como scatter_matrix.png
21 #         ")
22 #     plt.close()
23
24 def correlation_matrix(df: pd.DataFrame):
25     columns = [
26         "DepDelay",
27         "Month",
28         "DayOfMonth",
29         "DayOfWeek",
30         "CRSDepTime",
31         "CRSArrTime",
32         "Distance"
33     ]
```

2.1. CÓDIGOS

```

32     corr = df[columns].corr()
33     plt.figure(figsize=(9, 7))
34     plt.imshow(corr)
35     plt.colorbar()
36     plt.xticks(range(len(columns)), columns, rotation=45, ha="
        right")
37     plt.yticks(range(len(columns)), columns)
38     plt.title("Matriz de correlación")
39     plt.tight_layout()
40     plt.savefig("correlation_matrix.png", dpi=DPI)
41     plt.close()
42
43
44 def delays_hist(df:pd.DataFrame):
45     print("Generando histograma de DepDelay...")
46     plt.figure(figsize=(10, 6))
47     bins = int(np.sqrt(len(df)))
48     plt.hist(df["DepDelay"], bins=100, range=(-25, 150))
49     plt.xlabel("Retraso en la salida (minutos)")
50     plt.ylabel("Frecuencia")
51     plt.title("Distribución de los retrasos en la salida")
52     plt.savefig("depdelay_histogram.png", dpi=DPI)
53     print(" Grafico generado! guardado como depdelay_histogram.
        png")
54     plt.close()
55
56
57 def delays_vs_distance(df:pd.DataFrame):
58     print("Generando scatter plot de Distance vs DepDelay...")
59     plt.figure(figsize=(10, 6))
60     plt.scatter(df["Distance"], df["DepDelay"], s=1, alpha=0.2)
61     plt.xlabel("Distancia (millas)")
62     plt.ylabel("Retraso en la salida (minutos)")
63     plt.title("Relación entre distancia y retraso")
64     plt.savefig("distance_vs_depdelay.png", dpi=DPI)
65     print(" Grafico generado! guardado como distance_vs_depdelay.
        png")
66     plt.close()
67
68
69 def delays_vs_deptime(df:pd.DataFrame):
70     print("Generando scatter plot de DepTime vs DepDelay...")
71     plt.figure(figsize=(10, 6))
72     plt.scatter(df["CRSDepTime"], df["DepDelay"], s=1, alpha=0.2)
73     plt.xlabel("Hora de salida programada (minutos desde
        medianoche)")
74     plt.ylabel("Retraso en la salida (minutos)")
75     plt.title("Relación entre hora programada y retraso")
76     plt.savefig("departure_time_vs_depdelay.png", dpi=DPI)
77     print(" Grafico generado! guardado como
        departure_time_vs_depdelay.png")

```

2.1. CÓDIGOS

```

78     plt.close()
79
80
81 def delays_by_month(df:pd.DataFrame):
82     plt.figure(figsize=(10, 6))
83     df.boxplot(column="DepDelay", by="Month")
84     plt.xlabel("Mes")
85     plt.ylabel("Retraso en la salida (minutos)")
86     plt.title("Retraso en la salida por mes")
87     plt.suptitle("")
88     plt.savefig("depdelay_by_month.png", dpi=DPI)
89     plt.close()
90
91
92 def delays_by_week(df:pd.DataFrame):
93     plt.clf()
94     plt.figure(figsize=(10, 6))
95     df.boxplot(column="DepDelay", by="DayOfWeek")
96     plt.xlabel("Día de la semana")
97     plt.ylabel("Retraso en la salida (minutos)")
98     plt.title("Retraso en la salida por día de la semana")
99     plt.suptitle("")
100    plt.savefig("depdelay_by_dayofweek.png", dpi=DPI)
101    plt.close()
102
103
104 def mean_delay_by_month(df: pd.DataFrame):
105     monthly_mean = df.groupby("Month")["DepDelay"].mean()
106     plt.figure(figsize=(10, 6))
107     # plt.bar(monthly_mean.index, monthly_mean.values)
108     plt.plot(monthly_mean.index, monthly_mean.values, marker="o")
109     plt.xlabel("Mes")
110     plt.ylabel("Retraso promedio (minutos)")
111     plt.title("Retraso promedio en la salida por mes")
112     plt.xticks(range(1, 13))
113     plt.grid()
114     plt.savefig("mean_depdelay_by_month.png", dpi=DPI)
115     plt.close()
116
117
118 def mean_delay_by_week(df: pd.DataFrame):
119     weekly_mean = df.groupby("DayOfWeek")["DepDelay"].mean()
120     plt.figure(figsize=(10, 6))
121     plt.plot(weekly_mean.index, weekly_mean.values, marker="o")
122     plt.xlabel("Día de la semana")
123     plt.ylabel("Retraso promedio (minutos)")
124     plt.title("Retraso promedio en la salida por día de la semana"
125             )
126     plt.xticks(range(1, 8))
127     plt.grid()
128     plt.savefig("mean_depdelay_by_weekly.png", dpi=DPI)

```

2.1. CÓDIGOS

```

128     plt.close()
129
130
131 def mean_delay_by_hour(df: pd.DataFrame):
132     temp = df.copy()
133     temp["Hour"] = (temp["CRSDepTime"] // 60).astype(int)
134     hourly_mean = temp.groupby("Hour")["DepDelay"].mean()
135     plt.figure(figsize=(10, 6))
136     plt.plot(hourly_mean.index, hourly_mean.values, marker="o")
137     plt.xlabel("Hora de salida programada")
138     plt.ylabel("Retraso promedio (minutos)")
139     plt.title("Retraso promedio según hora de salida programada")
140     plt.xticks(range(25))
141     plt.grid()
142     plt.savefig("mean_depdelay_by_hour.png", dpi=DPI)
143     plt.close()

```

Código principal.

```

1 import subprocess # ELIMINAR AL FINAL, SÓLO ES PARA USO EN
  DESARROLLO
2 import time
3 import numpy as np
4 import pandas as pd
5 import matplotlib.pyplot as plt
6 from scipy.io import arff # Para leer el tipo de archivo en el que
  viene el dataset
7 from plotUtil import *
8
9 # FUNCION PARA SABER EL ESTADO DEL PROGRAMA, NO ES PARA USO FINAL
  EN DESARROLLO.
10 def program_status(message):
11     subprocess.run(["say", "-r", "250", message])
12
13
14 def delays_summary(df: pd.DataFrame):
15     print("\nResumen de DepDelay:")
16     percentiles = df["DepDelay"].quantile([0.01, 0.05, 0.25, 0.50,
17     0.75, 0.95, 0.99])
18
19     for percentile, value in percentiles.items():
20         print(f"{percentile * 100:.0f}%: {value:.2f} minutos")
21
22     print(f"Media: {df['DepDelay'].mean():.2f} minutos")
23     print(f"Desviación estándar: {df['DepDelay'].std():.2f} minutos")
24     print(f"Máximo retraso: {df['DepDelay'].max():.2f} minutos")
25     print(f"Mínimo retraso: {df['DepDelay'].min():.2f} minutos")
26
27 def etl_process(_plot_graphs: bool = False):
28     # ETL
29     ## Extracting

```

2.1. CÓDIGOS

```

29     program_status("Cargando datos...")
30     print("Cargando los datos...")
31     start = time.time()
32     arff_file = arff.loadarff('./
33         airlines_train_regression_10000000.arff')
34     df = pd.DataFrame(arff_file[0])
35     end = time.time()
36     print(f"Datos cargados en {round(end-start, 2)} segundos.")
37     program_status("Datos cargados.")
38     print("\nDatos generales:")
39     print(df.info())
40
41     ## Transform
42     print("\nExcluyendo columnas UniqueCarrier, Origin y Dest")
43
44     df = df.drop(columns=["UniqueCarrier", "Origin", "Dest"])
45
46     print("\nConvirtiendo CRSDepTime y CRSArrTime a minutos desde
47         la medianoche...")
48
49     def hhmm_to_minutes(time):
50         time = int(time)
51         hours = time // 100
52         minutes = time % 100
53         return hours * 60 + minutes
54
55     df["CRSDepTime"] = df["CRSDepTime"].apply(hhmm_to_minutes)
56     df["CRSArrTime"] = df["CRSArrTime"].apply(hhmm_to_minutes)
57
58     print("Conversión realizada.")
59
60     print("Datos generales:")
61
62     print(df.info())
63     print(df.describe())
64     print("Cantidad de valores nulos:")
65     print(df.isna().sum())
66     delays_summary(df)
67
68     if _plot_graphs:
69         # scatter_matrix_plot(df[["CRSDepTime", "CRSArrTime", "
70             Distance"]])
71         program_status("Comenzando con histograma de delays.")
72         delays_hist(df)
73         program_status("Comenzando con scatter plot de delays vs
74             distance.")
75         delays_vs_distance(df)
76         program_status("Comenzando con scatter plot de delays vs
77             deptime.")
78         delays_vs_deptime(df)
79         program_status("Comenzando con box plot de delays by month

```

2.1. CÓDIGOS

```

    .")
75     delays_by_month(df)
76     program_status("Comenzando con box plot de delays by week.
    ")
77     delays_by_week(df)
78     program_status("Comenzando con plot de promedio de retraso
        por hora.")
79     mean_delay_by_hour(df)
80     program_status("Comenzando con plot de promedio de retraso
        por mes.")
81     mean_delay_by_month(df)
82     program_status("Comenzando con matriz de correlación.")
83     correlation_matrix(df)
84     program_status("Comenzando con plot de promedio de retraso
        por día de la semana.")
85     mean_delay_by_week(df)
86
87     # Loading
88
89     Xtrain = df[]
90
91     Xtrain, Xtest, Xvalidation, ytrain, ytest, yvalidation = []
92
93     return df
94
95
96 def main():
97     plot_graphs = input("¿Desea generar gráficos a lo largo del
        programa? (y/n) ")
98     plot_graphs = True if plot_graphs == "y" else False
99     program_status("Iniciando ETL...")
100    df = etl_process(_plot_graphs=plot_graphs)
101    program_status("ETL Finalizado.")
102    program_status("Programa finalizado.")
103
104
105 if __name__ == '__main__':
106     main()

```